# PACT

Volume 2 - Engineering build specification and delivery plan

Objective: implement the complete authority -> intent -> liability quote -> policy -> reservation -> obligation -> execution -> evidence -> settlement -> receipt path on Monad. This document provides contract responsibilities, state invariants, canonical APIs, service boundaries, architecture decisions, test matrices and release gates. Use the companion file-by-file blueprint for exact filenames.

## 1. Architecture and threat boundary

```
Principal -> Console -> signed grant/approval -> Monad contracts
Agent -> TS/Python SDK or MCP -> Pact Engine -> [Account | Gateway]
Account -> Monad controlled assets; Gateway -> vaulted provider key
Counterparty -> independent proof verifier -> ObligationManager
Evidence -> named verifier -> SettlementVault -> ReceiptRegistry
Chain events -> indexer -> API read models -> Console / Explorer
```

The Engine is a decision and orchestration component; contracts enforce final asset and commitment constraints. The Gateway enforces external API policies only where it exclusively holds the upstream credential. The indexer is a projection, not an authority source. A chain hash proves content integrity, not the accuracy of a subjective verdict.

## 2. Canonical data models and units

```
Grant { id, principal, agentIdentity, operationalKey, parentId,
  policyHash, keyEpoch, validFrom, expiresAt, depth, status }
Intent { grantId, actionKind, chainId, target, selector, asset,
  amount, payloadHash, termsHash, nonce, deadline }
ExposureQuote { adapterId, adapterVersion, asset, cashNow,
  periodSpend, maximumLiability, reservationAmount, quoteExpiry }
AuthorityProof { grantId, principal, agent, policyHash, intentHash,
  termsHash, asset, liability, reservationId, nonce, expiresAt }
Obligation { parties, termsHash, grantId, maximumLiability,
  fundedAmount, deadline, verifierConfigHash, state }
Receipt { grantId, intentHash, decisionHash, approvalHash,
  obligationId, executionTxs, evidenceRoot, verdictHash,
  settlementTxs, outcome, disclosureVersion }
```

Amounts are unsigned integers in token base units. Each asset/ceiling pairing is separate. Token decimals and addresses are validated at configuration time. Do not compare USD labels to token units, or count escrow deposits as extinguishing gross future liability. Use signed EIP-712 messages with domain separation, unique nonces and expiry; support ERC-1271 principals. Store a canonical version number for every hashable schema.

## 3. Commitment accounting and concurrency

```
available(grant, asset) = ceiling - allocatedToChildren
                        - reserved - activeCommitments
Require: available >= 0 at every affected ancestor.
Atomic reserve: validate ancestry + policy root + amount +
               expiry; update all affected capacity in one tx.
Atomic commit: consume exactly one live reservation -> obligation.
Release: only explicit cancellation/expiry/settlement transitions.
```

A child allocation reduces the parent's free capacity. A child reservation must respect child allocation and every ancestor. Revocation blocks new operations but does not erase active commitments. A completed installment reduces the remaining unpaid obligation only if the terms permit it; a per-period spending counter increases on actual spend. Rolling windows have an explicit epoch and reset rule. Avoid double counting delegated and ancestor usage in displayed organization totals.

# 4. Contract responsibilities and interfaces

| Contract | Authoritative operations | Core negative tests |
|---|---|---|
| AuthorityRegistry | create root, delegate narrower scope, rotate key, revoke, check full ancestry | Forged signature, child scope expansion, revoked ancestor, stale key. |
| CommitmentLedger | allocate, reserve, commit, release, expire, read availability | Race/oversubscription, double consume, premature release, underflow. |
| PactAccount | verify action-bound permission and execute reviewed adapter | Unknown selector, changed calldata, replay, blanket approval. |
| ObligationManager | accept common terms, record funding/evidence/verdict/challenge/status | One-sided edit, invalid transition, expired terms, changed verifier. |
| SettlementVault | fund, hold, milestone payout, refund, recover under written rules | Double withdrawal, underfunded release, challenge bypass. |
| ReceiptRegistry | hash-linked event anchors and terminal receipt | False settlement, missing ancestor, altered evidence hash. |

Use narrow capabilities between contracts; neither the web backend nor an administrator can arbitrarily increment capacity or force settlement. Define upgrade and pause rights before deploying funds. Emit indexed event IDs and versioned hashes sufficient to rebuild the indexer.

# 5. Authorization engine algorithm

```
function authorize(intent, grant, policy, chainSnapshot):
  validate identity, nonce, signature, chain, deadline, ancestry
  adapter = registry.findExact(intent.actionKind, target, selector)
  if absent: DENY(UNSUPPORTED_ACTION)
  quote = adapter.boundWorstCase(intent, snapshot)
  if quote stale/unbounded: DENY(UNBOUNDED_EXPOSURE)
  simulate supported onchain call; record simulation block
  evaluate access, financial, time, delegation, economic rules
  if any hard fail: DENY(reasons)
  if approval threshold hit: REQUIRE_APPROVAL(exact quote/hash)
  else: ALLOW(exact quote/hash, short expiry)
  reserve onchain; account revalidates before execution
```

Policy compilation uses explicit precedence: deny overrides approval, approval overrides allow. The action adapter exposes target-specific bounded financial effects. Simulation alone does not prove safety because another transaction, oracle update or opaque downstream call can change the result. Requote when data or terms change.

# 6. Action adapters and financial semantics

| Adapter | Exposure calculation | Execution condition |
|---|---|---|
| ERC-20 transfer | cashNow = amount; liability = amount; optional explicit fee bound. | Token/recipient/amount match signed intent. |
| Fixed-price agreement | Maximum payment including explicit cancellation amount; escrow shown separately. | Terms, verifier and deposit match signed terms hash. |
| Recurring agreement | Conservative sum of payable periods plus non-overlapping penalties. | Term and upper bound fixed; amendments require new quote. |
| API provider | Provider published/request cap reserved before call, reconciled after result. | Vault only forwards supported bounded endpoint. |
| Leverage venue | Only exact reviewed protocol whose worst-case exposure is defined. | Disabled in MVP; unknown protocol always denied. |

# 7. Agreement, verifier and settlement state machines

```
PROPOSED -> RESERVED -> ACTIVE -> SUBMITTED -> VERIFIED
                                  |            |
                               CANCELLED    DISPUTED -> RESOLVED
VERIFIED / RESOLVED -> SETTLED; reservation -> EXPIRED if unused
```

PROPOSED carries no third-party guarantee. RESERVED has locked economic capacity and short expiry but may lack provider signature. ACTIVE requires signatures and required funding. SUBMITTED records evidence hash and deadline. VERIFIED records a named verifier result. DISPUTED prevents contested payouts until resolution. SETTLED is terminal for completed milestones, with receipt anchors. Terms define partial settlement, timeout and cancellation behavior. Every transition emits a typed event.

# 8. Proof and receipt verification

• Authority proof verification checks chain ID, verifying contract, signatures, current grant ancestry, key epoch, terms/intents, reservation live status, liability and expiry. A mere read of available capacity is a non-binding snapshot.

• Obligation proof checks both party signatures on identical terms, authority reservation consumption and current onchain obligation state.

• Receipt verification reconstructs hash-linked events and settlement transactions, marks private evidence as disclosed/undisclosed, and labels outside attestations.

• Independent verifier must run from public RPC and supplied proof bundle if Pact API/indexer is unavailable.

# 9. External API Gateway and secret lifecycle

Authenticate scoped agent -> parse supported endpoint -> check host/method/payload/asset/cost -> reserve bounded usage -> access KMS-backed upstream credential -> forward without redirect drift -> record upstream idempotency key and response digest -> reconcile actual usage -> release unused capacity. Block SSRF, DNS rebinding, redirect to unapproved host, header injection, oversize body and provider response leakage. Vault access is service-scoped and audited; never expose secret to SDK/MCP/browser.

# 10. APIs and developer contracts

| Endpoint / method | Request and result |
|---|---|
| POST /v1/intents/quote | Intent -> quote, assumptions, expiry, adapter version. |
| POST /v1/intents/authorize | Intent -> DENY / REQUIRE_APPROVAL / ALLOW, reasons, policy hash. |
| POST /v1/approvals | Exact decision and signed human approval -> pending/confirmed. |
| POST /v1/reservations | Approved intent -> onchain reservation ID and proof bundle. |
| POST /v1/obligations | Proof and both signed terms -> obligation ID, tx state. |
| POST /v1/obligations/:id/evidence | Signed evidence manifest -> immutable hash, pending verifier state. |
| POST /v1/obligations/:id/challenge | Bound challenge -> hold and resolver process. |
| POST /v1/obligations/:id/settle | Eligible state -> transaction receipt and final status. |
| GET /v1/receipts/:id | Linked proof/attestation bundle subject to disclosure policy. |

All mutation endpoints require tenant-scoped auth, schema validation, idempotency key, audit correlation ID and signatures where applicable. Return distinct request-accepted, chain-pending, chain-confirmed and final-settlement states. Never allow an HTTP success status to imply funds settled.

### SDK and MCP

TypeScript and Python SDKs implement identical canonical hashing and typed errors. MCP tools expose quote, authority lookup, proposed obligation, approval request, approved execution, evidence submission and receipt lookup. MCP server authorization binds a client to agent/principal/grant; the LLM prompt never supplies missing economic consent. Protect per-tool disclosure and cross-tenant boundaries.

# 11. Persistence, indexing and infrastructure

• PostgreSQL holds encrypted offchain policy versions, approval queue, idempotency records and read projections; chain events remain authoritative.

• Object store holds encrypted evidence blobs with immutable content hashes and access policy.

• Indexer handles reorg/rollback and replay; records exact chain cursor and contract ABI version.

• Secrets use dedicated vault/KMS; staging and production credentials are separate.

• Services run behind TLS with request limits, health/readiness probes, structured logs, metrics and queue retry/dead-letter handling.

# 12. Security verification plan

| Test class | Must prove |
|---|---|
| Solidity unit/fuzz | Attenuation, revocation, no replay, no over-commitment, exact authorization, escrow conservation. |
| Stateful invariants | Random child/reserve/commit/release sequences never create negative capacity or duplicate payout. |
| Gateway adversarial | No raw secret leak, unauthorized host/method, redirect/SSRF or repeated billing. |
| Cross-language | TS/Python/Solidity hashes and signed payloads agree across fixtures. |
| Integration | Real local chain plus Engine creates valid proof and obligation; failure recovers reservation safely. |
| Browser e2e | Agent denied bad action, human approves revised offer, provider verifies and gets correct settlement. |
| Independent proof | Turn Pact API off; verifier still detects tampered proof/receipt from chain. |

Before production funds: independent smart contract audit, external penetration review of Gateway and Console, incident drills, privileged-role inventory, withdrawal/pause drills and written disclosure of trust assumptions. A testnet demo is not a security audit.

# 13. Operations and recovery

| Failure | Expected response |
|---|---|
| RPC unavailable | Fail closed for privileged actions; queue/retry read; never claim confirmed state. |
| Indexer behind or reorg | Show lag; validate authorization against RPC, rollback projections and replay. |
| Approval expires | Reject execution and request new signed approval for exact current terms. |
| Reservation expires | Reject proof; explicit safe reclaim path; do not activate agreement. |
| Gateway provider times out | Check idempotency/provider status before retry; hold maximum reservation until reconciled. |
| Verifier offline | Use terms-defined timeout and challenge/resolver path, not operator guess. |
| Compromised agent key | Rotate/revoke grant; preserve obligations; review audit and provider credential use. |
| Vault emergency | Pause new execution while preserving defined safe exits and dispute rights. |

# 14. Delivery plan and dependencies

| Stage | Implementation | Exit gate |
|---|---|---|
| 0 Decisions | Network/asset, verifier, funding, custody, dispute and upgrade ADRs. | Exact addresses, trust model and approved assumptions. |
| 1 Schema | Canonical versioned types, EIP-712 vectors and cross-language fixtures. | Hashes match in TS/Python/Solidity. |
| 2 Contracts | Authority, ledger, controlled account, obligation, vault, receipts. | Unit, fuzz and invariant suite green. |
| 3 Engine | Exposure adapters, simulation, policy, approvals, reservations. | Demonstrate three DENY classes and bounded approval. |
| 4 Services | API, Gateway, secrets, indexer, evidence storage. | No secret bypass; recover from retries/outages. |
| 5 Developer/UI | SDKs, MCP, Console, Explorer and independent proof package. | New agent integrated and full workflow visible. |
| 6 Launch | Testnet deployment, audit review, demo recording, code provenance. | Reproducible submission with no critical unresolved issue. |

# 15. Repository and handoff requirements

```
apps/{api,gateway,mcp,console,explorer,indexer}
packages/{contracts,schemas,engine,sdk-ts,sdk-python,proofs}
examples/procurement-agent
infra/{docker,deploy,monitoring}
tests/{integration,e2e,invariants}
docs/{protocol,api,operations}
```

For each package include README, build/test command, environment example, typed interfaces, error contracts, fixture data and relevant test. CI must build all packages and run the vertical slice. The separate Pact File-by-File Implementation Blueprint enumerates exact target files and completion checks.

# 16. Final acceptance and hard stop rules

• No claim of enforcement if the agent also has an unrestricted principal wallet key or real upstream API credential.

• No ALLOW for unsupported actions or unbounded liabilities; require a reviewed adapter.

• No counterparty guarantee based solely on an available-balance snapshot; use a live reservation.

• No unilateral mutation of already signed economic terms.

• No receipt claiming final settlement while a chain transaction or dispute is pending.

• No production custody of real value before security review and documented operational controls.

# 17. References and current-state checks

Official sources: Monad https://docs.monad.xyz/ and https://docs.monad.xyz/developer-essentials/changelog ; Metropolis https://monad.xyz/metropolis ; ERC-8004 https://eips.ethereum.org/EIPS/eip-8004 ; ERC-7715 https://eips.ethereum.org/EIPS/eip-7715 ; EIP-712 https://eips.ethereum.org/EIPS/eip-712 ; AP2 https://ap2-protocol.org/ap2/agent_authorization/ ; MCP https://modelcontextprotocol.io/specification/2026-07-28 . Verify deployed token, contract addresses, SDK versions, hackathon and bounty rules at implementation time.